WINGMAN · DEPLOYMENT

# From prototype to the App Store

What it takes to put Wingman in front of real people — and the two things in the code today that have to change before anyone else's data touches it.

16 August 2026  Tide Capital  Branch `wingman-v3`

---

## Where it stands

Wingman today is a client-only app: a Vite + React bundle with every engine written as a pure, tested function and every byte of state in the browser. That is a good place to be — the hard logic is done and provable — but nothing about it is multi-user yet.

● ALREADY BUILT  ● YOUR WORK  ● EXTERNAL DEPENDENCY

— the colour tells you who has to move.

| LAYER | TODAY | STATUS |
|---|---|---|
| Matching engine | Pure TypeScript, no clock, no randomness, property-tested | ● BUILT |
| Privacy engine | Two-rule model, disclosure ladder, redaction — all client-side | ● MOVE TO SERVER |
| Airport data | 3,270 airports worldwide with derived IANA timezones, bundled | ● BUILT |
| Flight schedules | Bundled + synthetic fallback; live providers behind an interface | ● NEEDS A KEY + PROXY |
| Verification stamps | Mocked behind a provider contract | ● CRIIPTO / OAUTH |
| Persistence | `localStorage` / IndexedDB, one device, one person | ● NEEDS POSTGRES |
| Accounts | None. There is one seeded "you". | ● NEEDS AUTH |
| Native shell | None. | ● APPLE + A MAC |

# Two problems to fix before launch

Both are invisible while the app is single-player and both become serious the moment there is a server. Neither is difficult; both are much cheaper to fix now than after a schema exists.

FINDING 01 · SECURITY

## Client-side privacy is not privacy

The `audience` and `seeking` rules run in the browser. That is correct for a prototype, where the browser holds all the data anyway. With a real API it stops being a privacy control and becomes a *suggestion*: anyone can call the endpoint directly, skip the client entirely, and read the rows the rules were supposed to withhold.

**The fix:** visibility becomes a database guarantee via Postgres row-level security. The client keeps doing redaction for presentation; the database decides what rows exist at all. Detail in Privacy in the database.

FINDING 02 · SECRETS

### Anything named `VITE_*` ships to the browser

Vite inlines every `VITE_`-prefixed variable into the bundle at build time. The flight API keys in `.env.example` — `VITE_AERODATABOX_KEY` and the rest — would be readable in DevTools by anyone who opens the site, and billable to you until the quota burns.

**The fix:** the key moves to a Supabase Edge Function that proxies the provider. The client calls your function; your function holds the secret. The existing resolver chain does not change shape — only which side of the wire it runs on.

## The target shape

Three pieces. The web build and the iOS build come from the same bundle, which is the entire argument for Capacitor over a rewrite.

### Web — Vercel

Static SPA from `npm run build`. Preview deployment per branch, custom domain, and the marketing site on the same project. This part is close to free and close to instant.

### iOS — Capacitor

Wraps the existing `dist/` in a native shell with real native APIs for location, camera and deep links. Not React Native: no rewrite, no second codebase, no re-testing the engines.

### Backend — Supabase

Postgres with row-level security, Auth, Storage for photos, Realtime for requests, Edge Functions for the flight proxy, `pg_cron` for expiring requests and guardian sessions.

### Why not React Native or a PWA

**React Native** would mean rebuilding every screen and re-verifying engines that are already tested — months of work to arrive where you already are. **A PWA** is not an App Store path at all: Apple does not list them, and your go-to-market is a conference handing out a link to an app people expect to find in the store. Capacitor is the only route that preserves the work.

# The roadmap

Ordered by dependency, not by preference. Phases 1–3 unblock everything else; 4 and 5 can run in parallel once accounts exist. Phases marked in ember are on the critical path to a TestFlight build.

P1

### Web deploy

● 1 DAY

Point Vercel at the repo, set the build to `npm run build` and the output to `dist`. Hash routing already works without rewrite rules, so there is nothing to configure. You get a shareable URL the same afternoon — useful immediately for showing schools and conferences.

P2

### Supabase project, schema, RLS

● 2–3 WEEKS

The big one. Tables for people, trips, requests, meets, ratings, circles and memberships; auth wired to the existing profile slice; and the privacy model expressed as RLS policies. Do RLS *as* you write the schema, never after — retrofitting policies onto a live schema is how visibility bugs ship.

P3

### Apple Developer + a Mac

● 1–2 WEEKS, MOSTLY WAITING

$99/year, and enrolment as a company needs a D-U-N-S number, which is the slow part — start it before you need it. You also need macOS to build: either a Mac, or a hosted macOS runner (GitHub Actions, Codemagic). **There is no way to produce an iOS build on Windows.** This is the single hardest dependency in the plan and the reason it starts early.

P4

### Capacitor shell + TestFlight

● 1–2 WEEKS

Add the iOS platform, set the purpose strings, wire deep links for BankID's return trip, archive, and upload. First TestFlight build is usually the same day the certificates finally behave.

P5

### Real verification providers

● 2–4 WEEKS

BankID through Criipto or Signicat; LinkedIn, Instagram and Facebook through OAuth. The provider contract already exists, so each one is a file and a registry line — the time goes to their approval processes, not your code. Meta's app review for Instagram is the slowest.

P6

### Trust & safety before review

● 1–2 WEEKS

Apple will not pass a user-content app without moderation, reporting, blocking and account deletion. Block and report already exist in the request flow; you need the moderation queue, the 24-hour response commitment, and in-app account deletion.

P7

### Submit

● 1–3 WEEKS OF REVIEW

Screenshots, privacy nutrition label, age rating, and a demo account that reviewers can actually sign into — an app that shows an empty board to a reviewer with no trip gets rejected under 2.1 for looking broken. Seed the demo account with a trip.

## Privacy in the database

The engine's central rule is that visibility is mutual: A sees B only if B's audience admits A *and* A's seeking wants B. That rule has to hold at the row level, so the API cannot return a person the policy forbids even to a caller who bypasses the app entirely.

```
-- Both directions, in the database. The client no longer decides.
create or replace function can_see(viewer uuid, subject uuid)
returns boolean language sql stable security definer as $$
  select admits(subject, viewer)   -- subject's audience admits viewer
     and wants (viewer,  subject)  -- viewer's seeking wants subject
     and not blocked(viewer, subject);
$$;

alter table people enable row level security;

create policy people_are_mutually_visible
  on people for select
  using ( id = auth.uid() or can_see(auth.uid(), id) );
```

- **Redaction stays on the client.** The ladder decides which *fields* to show at each disclosure level; RLS decides which *rows* exist. Different jobs — keep them separate.
- **Never persist derived state** — the same rule as today. No materialised match lists, no cached visibility. A policy change has to invalidate instantly, and it only does if nothing stale was stored.
- **Index for the policy.** RLS predicates run per row; `can_see` over an unindexed table will be the first thing that gets slow. Test it against 10,000 seeded people before you believe it.
- **Port the property tests.** The fast-check properties that prove `mutual === aSees && bSees` should run against the SQL too, via pgTAP. The engine and the database must agree, or the client shows people the server would refuse.

# Verification in production

Every provider is already mocked behind the same contract, so this phase is procurement rather than engineering.

| STAMP | HOW | NOTE |
|---|---|---|
| BankID (NO) | `Criipto / Signicat` | Nobody integrates BankID directly — you go through a broker. Criipto is self-serve; Signicat is enterprise and quoted. |
| LinkedIn | `OAuth 2.0 · OIDC` | "Sign In with LinkedIn using OpenID Connect" gives name and email. The company field needs the deeper product. |
| Facebook | `OAuth 2.0` | Meta app review required before public use. |
| Instagram | `Instagram Basic Display` | Slowest approval of the three. Handle-only verification, which is all the stamp claims. |
| Email domain | `Supabase Auth OTP` | Cheapest and most valuable one you have — it powers circle admission, which is what schools are buying. |

APPLE · GUIDELINE 4.8

## Third-party login obliges Sign in with Apple

Offering LinkedIn or Facebook login means you must also offer an equivalent privacy-preserving option — one that limits data collection to name and email, allows the email to be kept private, and does not track for advertising. Sign in with Apple qualifies by construction. Supabase supports it natively; budget an afternoon, not a sprint.

# The App Store review gauntlet

An app for meeting strangers gets read carefully. Each item below is a rejection reason with a known fix, and several are already half-built.

### 1.2 User-generated content ● PARTLY BUILT

Requires four things: a method for filtering objectionable content, a mechanism for users to report it with a timely response, the ability to block abusive users, and published contact information.

**You have** blocking and reporting — the deny flow's "this made me uncomfortable" already blocks and flags. **You need** a moderation queue behind it, a published commitment to act within 24 hours, and a support address in the listing.

### 5.1.1(v) Account deletion ● NOT BUILT

Any app that lets a user create an account must let them delete it from inside the app — not by emailing support, not by a web form. This is a hard, automated check and a common first rejection.
With Supabase it is one Edge Function that cascades the delete and revokes the session. Worth doing properly: the guardian sessions and location pings must go too.

### 5.1.1 Location purpose strings ● NEEDS CARE

Guardian tracking needs `NSLocationAlwaysAndWhenInUseUsageDescription`, which draws more scrutiny than the foreground string. The purpose text has to say plainly what it does and who sees it — "so a person you choose can see where you are during a meet, until it ends" — not "to improve your experience".
The design helps here: the session is time-boxed and scoped, and it expires. Say that in the string.

### 2.1 Completeness — the demo account ● EASY TO MISS

Reviewers get a fresh account with no trip, so they see an empty board and reject the app as non-functional. Provide credentials in App Review notes for an account that already has a trip and a populated board, and say in the notes that content depends on having a flight.

**Age rating** ● EXPECT 16+ OR 18+

Apps built around meeting strangers land high on the scale, and anything read as dating goes to 18+. Wingman's non-dating framing is genuine, so make it legible *in the listing itself* — the screenshots, the description and the first-run screen should all show the professional and social framing, not just the card stack. This matters commercially: an 18+ rating complicates a university pilot where some students are 17.

**App Privacy nutrition label** ● STRAIGHTFORWARD

You declare what you collect and whether it is linked to identity. Wingman collects precise location, contact info, identifiers and user content. The declaration is easy — the useful exercise is checking that the answer is as small as you think it is. Data minimisation is a feature you can sell to a school.

# What it costs

Almost nothing until there are real users, then it steps rather than curves. BankID is the item that dominates, and it is charged per verification — so verification stays opt-in, not a signup requirement.

| LINE ITEM · USD/MO | 1K | 10K | 100K |
|---|---|---|---|
| Apple Developer Program | $8 | $8 | $8 |
| Vercel Pro | $20 | $20 | $20 |
| Supabase Pro | $25 | $25 | $60 |
| Flight data | $10 | $22 | $220 |
| BankID (Criipto) | $113 | $164 | $677 |
| Moderation tooling | $0 | $50 | $300 |
| **Total** | **$176** | **$289** | **$1,285** |
| Without BankID | $63 | $125 | $608 |

Assumes 2 flight lookups per active user per month; Criipto at roughly €99 base plus €0.35 per verification, billed once per person — so only new joiners are charged, taken as 1.5% of the active base each month; and Supabase compute stepping up above 50k MAU. Apple is $99/year shown monthly. The last row is the number that matters early: BankID is more than half the bill and nothing needs it until a customer asks.

# Timeline

Roughly ten to fourteen weeks to a submitted build, assuming the schema work and the Apple enrolment run in parallel. The Mac is the constraint that decides the date.

| WEEKS | WHAT HAPPENS | BLOCKS |
|---|---|---|
| 0–1 | Vercel live. Apple enrolment and D-U-N-S started. | Everything downstream of Apple |
| 1–4 | Supabase schema + RLS + auth. Flight key behind an Edge Function. | Multi-user anything |
| 3–5 | Capacitor shell, purpose strings, deep links. First TestFlight. | Needs a Mac |
| 4–8 | Verification providers. Meta review runs on their clock. | Circle admission needs email only |
| 6–9 | Moderation queue, account deletion, support address. | Submission |
| 9–11 | Screenshots, privacy label, demo account, submit. | — |
| 11–14 | Review, near-certain first rejection, resubmit. | — |

Plan for one rejection. Almost every social app gets one, and 1.2 or 5.1.1(v) is usually the reason — both are on this list precisely so they are not a surprise.

# Schools and conferences

If circles are what you sell, the circle stops being a feature and becomes the product's commercial unit. That has consequences for what gets built first.

### A circle is three things

A verified email domain, a crest, and — new — a date range. Schools are permanent circles; conferences are time-boxed ones that stop matching when the event ends. Same primitive, one extra field.

### The wedge

Every conference already has an app, and nobody opens it after collecting the schedule. Wingman is not competing with it — it is the "who should I meet here" layer, which is the thing attendees actually came for and no agenda app does.

### Why it is easier to sell than consumer

A closed loop solves cold-start on day one: 400 verified attendees in one terminal beats 400 strangers spread across a country. The organiser supplies the density you would otherwise have to buy.

### Pricing

Per event for conferences, per seat per year for schools. The verified-domain admission is the defensible part — it is why an INSEAD circle is actually INSEAD, and it is the thing a spreadsheet cannot replicate.

### What this changes in the build order

Email-domain verification stops being one stamp among five and becomes the first one to ship — it is the gate for circle admission and therefore for the entire pilot motion. BankID can wait; a business school does not need government ID to believe an `@insead.edu` address. That reordering saves several weeks and a monthly bill before there is revenue.

## Risks

- **Moderation is a staffing cost, not a feature.** Apple's "timely response" is a commitment by a person. At pilot scale it is minutes a day; plan who owns it before you promise it in the listing.
- **BankID costs money before it earns any.** Roughly €99/month to have it at all. Defer until a customer asks for it by name — email-domain verification carries the school pilot on its own.
- **Apple may read the app as dating regardless of framing.** Mitigate in the listing, not in an appeal: screenshots and description that lead with the professional and social use, and a first-run screen that says what this is.
- **The Mac dependency is absolute.** No Windows path exists to an iOS build. Decide between buying a Mac and renting a CI runner in week one, not week five.
- **Photos become user uploads.** The current portraits are generated likenesses, which is the right call for a prototype. Real uploads bring image moderation into

scope — the same queue as 1.2, so build it once.

- **RLS performance is the sleeper.** The policy runs per row on every read. It will be fine at pilot scale and it is the first thing that breaks at ten thousand people. Load-test it while the schema is still cheap to change.

---

Wingman · deployment roadmap · 16 August 2026 · prepared for Shun Gong, Tide Capital